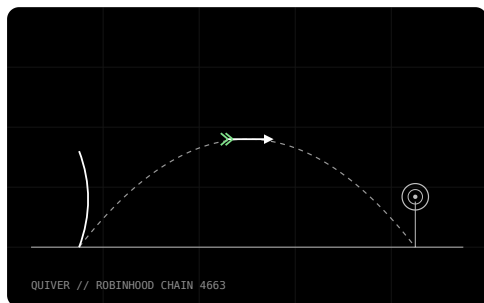


QUIVER

one v4 pool. 4663 arrows.

Guia de Lançamento Detalhado

Como colocar os contratos e o site no ar na Robinhood Chain, do zero ao pós-lançamento — cada etapa com comandos, saídas esperadas e o que fazer quando algo falha.



Versão 1.0 · Julho 2026 · repositório Pinguim306/v4-hook

Tempo total estimado: ~2h de execução (fora espera de bridge) · Custo on-chain: < US\$ 1 em gás

Sumário

1. **Parte 0** — Preparação: o que ter antes de começar
2. **Parte 1** — Validação completa (sem gastar nada)
3. **Parte 2** — Decidir o preço de lançamento
4. **Parte 3** — Deploy dos contratos
5. **Parte 4** — Seed: o lançamento (irreversível)
6. **Parte 5** — Publicar o site
7. **Parte 6** — Pós-lançamento (primeiras 48h)
8. **Apêndice A** — Solução de problemas
9. **Apêndice B** — Cartão de referência rápida

Como usar este guia

Execute as partes em ordem. Cada etapa termina com um bloco verde “**✓ Critério de sucesso**” — não avance sem cumpri-lo. Blocos laranja são avisos de irreversibilidade ou segurança. Comandos assumem terminal `bash/zsh` (Linux ou macOS) na raiz do repositório.

Parte 0 Preparação: o que ter antes de começar

0.1 — Instalar as ferramentas

Você precisa de **git**, **Node.js 20+** e **Foundry** (forge/cast).

```
# verificar o que já existe
git --version && node --version && forge --version

# instalar Foundry (se faltou)
curl -L https://foundry.paradigm.xyz | bash
foundryup

# instalar Node 20 via nvm (se faltou)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
nvm install 20
```

✓ Critério de sucesso

`forge --version` imprime uma versão ≥ 1.0 e `node --version` imprime v20 ou superior.

0.2 — Criar a carteira de deploy

⚠ Segurança

Use uma carteira **nova, exclusiva para este deploy**. A chave privada vai ficar num arquivo `.env` local — nunca use sua carteira principal, nunca commite o `.env`, e esvazie a carteira depois do lançamento.

```
# gera um par novo — anote os dois valores com cuidado
cast wallet new
Successfully created new keypair.
Address:      0xSEU_ENDERECO_DE_DEPLOY
Private key:  0xSUA_CHAVE_PRIVADA
```

Guarde a chave num gerenciador de senhas. O endereço será o `H00K_OWNER` — o único que pode chamar `seed()` uma vez (e esse poder se autodestrói dentro do próprio seed).

0.3 — Colocar ETH na Robinhood Chain

Quanto: ~0,05 ETH é folga confortável (o gás real de deploy+seed é < US\$ 1; o resto volta).

Como:

1. Adicione a rede à sua carteira (MetaMask → Add network → manualmente):

Campo	Valor
Network name	Robinhood Chain
RPC URL	https://rpc.mainnet.chain.robinhood.com
Chain ID	4663
Currency symbol	ETH
Block explorer	https://robinhoodchain.blockscout.com

2. Faça o bridge de ETH (de Ethereum ou Arbitrum) pela bridge oficial listada em docs.robinhood.com/chain ou pelo portal portal.arbitrum.io (Robinhood Chain é uma chain Arbitrum Orbit).
3. Envie o ETH bridgeado para o endereço de deploy criado em 0.2.

✓ Critério de sucesso

```
cast balance 0xSEU_ENDERECO --rpc-url https://rpc.mainnet.chain.robinhood.com mostra saldo > 0.
```

0.4 — Clonar o repositório

```
git clone https://github.com/Pinguim306/v4-hook.git
cd v4-hook
```

Estrutura relevante: `contracts/` (Foundry), `web/` (site), `docs/` (este guia, runbook, endereços).

0.5 — Decidir onde o site vai morar

Opção	Esforço	O que precisa
A. Cloudflare Pages / Vercel / Netlify (recomendado para começar)	Baixo	Conta gratuita no serviço
B. GitHub Pages	Baixo	Nada além do repo
C. IPFS + ENS (estilo Prism, imutável)	Médio	Nome ENS (~US\$ 5/ano) + conta num pinning service (Pinata / web3.storage)

Dá para começar com A e adicionar C depois — o build é o mesmo.

Parte 1 Validação completa (sem gastar nada)

Nada nesta parte custa gás. O objetivo é sair dela com **certeza técnica** de que o lançamento vai funcionar.

1.1 — Instalar as dependências Solidity

```
cd contracts
./bootstrap.sh
```

O que faz: baixa as 5 bibliotecas (forge-std, solady, v4-core, v4-periphery, permit2) em **commits pinados** — a árvore de dependências é 100% reproduzível.

```
Installing forge-std in .../lib/forge-std ...
  Installed forge-std
  ...
Dependencies installed.
```

Se falhar: é rede (clones do GitHub). Rode de novo; o script pula o que já foi instalado.

1.2 — Compilar e rodar a suíte de testes completa

```
forge build
forge test -vv
```

O que faz: compila tudo com solc nativo (baixado automaticamente pelo Foundry) e roda:

- **22 testes unitários** (`test/QuiverUnit.t.sol`) — lógica do hook com mocks;
- **Suíte e2e local** (`test/e2e/QuiverE2E.t.sol`) — deploy de um PoolManager + PositionManager reais localmente, com swaps de verdade;
- O fork test aparece como `SKIP` (ele só roda com `--fork-url` — etapa 1.4).

```
Ran N test suites: todos os testes PASS, 1 skipped (fork)
```

✓ Critério de sucesso

Nenhum FAIL. **Esta é a primeira execução da suíte e2e com solc nativo** — se algo falhar aqui, pare e reporte a saída completa antes de qualquer deploy.

1.3 — Pré-flight: validar a infraestrutura v4 on-chain

```
./preflight.sh
```

O que faz (7 checagens contra a chain real, só leitura):

1. Chain ID do RPC == 4663;
2. PoolManager 0x8366...0951 tem bytecode;
3. PositionManager 0x58da...4fA7 tem bytecode;
4. Permit2 0x0000...8BA3 tem bytecode;
5. POSM responde `nextTokenId()` — função que o `seed()` chama;
6. POSM aponta para o PoolManager informado (`poolManager()` — cross-check de que os endereços se pertencem);
7. Permit2 responde `DOMAIN_SEPARATOR()` .

```
== Robinhood Chain pre-flight ==  
✓ chain id is 4663  
✓ PoolManager has code (...) at 0x8366...  
✓ POSM answers nextTokenId()  
✓ POSM.poolManager() == PoolManager (cross-check)  
...  
== 7 passed, 0 failed ==
```

✓ Critério de sucesso

7 passed, 0 failed . Qualquer ✗ significa endereço errado ou RPC de outra rede — corrigir antes de continuar.

1.4 — Fork test: ensaio geral do lançamento

```
forge test --match-contract QuiverForkTest \  
--fork-url https://rpc.mainnet.chain.robinhood.com -vv
```

O que faz: cria um fork local da Robinhood Chain (cópia do estado real, sem gastar nada) e executa o ciclo de vida completo contra os contratos **reais** da Uniswap:

1. **seed** — inicializa a pool no PoolManager real e mint a posição LP no POSM real (exatamente as chamadas do lançamento);
2. **compra** — swap de ETH → QUIVER pelo PoolManager real mint Arrows para o comprador;
3. **fees** — `pokeFees()` colhe taxas da posição real; o Arrow acumula valor;
4. **claim** — pagamento chega ao dono;
5. **arte** — `tokenURI` resolve pelo mirror.

```
[PASS] test_Fork_FullLifecycle() (gas: ...)  
Suite result: ok. 1 passed; 0 failed
```

✓ Critério de sucesso

PASS. **Se este teste passa, o lançamento funciona** — ele elimina o único risco técnico restante (compatibilidade com a infra real da chain).

Parte 2 Decidir o preço de lançamento

É a **única decisão econômica** do lançamento, e é irreversível a partir do seed. Dois números no `.env` controlam tudo:

- **TICK_UPPER** — define o **preço inicial**: 1 QUIVER custa $1,0001^{(-\text{TICK_UPPER})}$ ETH. Tick maior = lançamento mais barato.
- **TICK_LOWER** — define o **teto de valorização** (o preço quando todo o supply tiver sido vendido): $1,0001^{(-\text{TICK_LOWER})}$ ETH.

Ambos precisam ser múltiplos de 200 (o script valida e reverte se não forem).

Preço inicial por QUIVER	FDV inicial (4663 tokens)	TICK_UPPER
~0,0002 ETH	~0,93 ETH	85200
~0,0005 ETH	~2,33 ETH	76000
~0,001 ETH ← recomendado	~4,70 ETH	69000
~0,002 ETH	~9,28 ETH	62200
~0,005 ETH	~23,3 ETH	53000
~0,01 ETH	~46,9 ETH	46000

Para **TICK_LOWER**, use **-887200** (o mínimo permitido) — teto de valorização astronomicamente alto, ou seja, sem teto prático. Recomendado.

Preço personalizado: para um alvo p (em ETH por QUIVER):

```
python3 -c "import math; p=0.001; t=-math.log(p)/math.log(1.0001); print(round(t/200)*200)"
```

Raciocínio

FDV inicial baixo deixa espaço para a valorização que recompensa os primeiros compradores — é o padrão dos fair launches bem-sucedidos (o Prism seguiu essa linha). Como o deployer não coloca ETH na pool, não há “preço de custo” a recuperar: o preço inicial é puramente uma escolha de curva.

Parte 3 Deploy dos contratos

3.1 — Configurar o `.env`

```
cd contracts
cp .env.example .env
# edite o .env com seu editor preferido:
```

Variável	Valor
RPC_URL	já preenchido (mainnet)
POOL_MANAGER / POSITION_MANAGER / PERMIT2	já preenchidos (validados na Parte 1)
HOOK_OWNER	endereço da carteira de deploy (0.2)
PK	chave privada dela
TICK_UPPER / TICK_LOWER	da Parte 2 (ex.: 69000 / -887200)
HOOK	deixe como está — só existe depois do deploy (3.2)

```
source .env
```

3.2 — Executar o deploy

```
forge script script/Deploy.s.sol:DeployQuiver \
  --rpc-url $RPC_URL --broadcast --private-key $PK
```

O que acontece internamente:

1. O script **minera um salt CREATE2**: testa salts até achar um endereço cujos 14 bits finais codificam exatamente a flag `AFTER_SWAP` (0x0040) — requisito do Uniswap v4 para o hook ser aceito. Leva de segundos a ~2 min de CPU local;
2. Deploya o `QuiverHook` por esse endereço via CREATE2 Deployer canônico (0x4e59...956C);
3. O construtor do hook: mint a os 4663 QUIVER para si, cria o `QuiverMirror`, aprova o Permit2/POSM (preparação do seed) e valida os flag bits do próprio endereço — se a mineração estivesse errada, **reverteria aqui**;
4. Confere que o endereço deployado == minerado.

```
Mined hook address: 0xAbC...0040
QuiverHook deployed: 0xAbC...0040
QuiverMirror (ERC721): 0xDeF...
```

✓ Critério de sucesso

As duas linhas finais com endereços. **Anote os dois**: cole `H00K=0x...` no `.env` e registre ambos em `docs/DEPLOYMENTS.md`.

Se falhar: deploy custa centavos — nenhum estado público foi criado. Corrija e rode de novo (um novo endereço será minerado; o anterior nunca foi usado e pode ser esquecido).

3.3 — Verificações imediatas (antes do seed)

```
export H00K=0x... # o endereço do deploy — ou source .env de novo
cast call $H00K "name()(string)" --rpc-url $RPC_URL
"Quiver"
cast call $H00K "SUPPLY()(uint256)" --rpc-url $RPC_URL
466300000000000000000000 [4.663e21]
cast call $H00K "seeded()(bool)" --rpc-url $RPC_URL
false
cast call $H00K "owner()(address)" --rpc-url $RPC_URL
0xSEU_ENDERECO_DE_DEPLOY
```

✓ Critério de sucesso

os quatro valores acima, exatamente.

3.4 — Verificar o código-fonte no Blockscout

Fazer isso **antes** do seed: transparência total antes do primeiro trade.

```
# o hook (constructor: poolManager, owner, posm, permit2)
forge verify-contract $HOOK src/QuiverHook.sol:QuiverHook \
  --verifier blockscout \
  --verifier-url https://robinhoodchain.blockscout.com/api \
  --constructor-args $(cast abi-encode \
    "constructor(address,address,address,address)" \
    $POOL_MANAGER $HOOK_OWNER $POSITION_MANAGER $PERMIT2)

# o mirror (constructor: hook) – pegue MIRROR da saída do deploy
forge verify-contract $MIRROR src/QuiverMirror.sol:QuiverMirror \
  --verifier blockscout \
  --verifier-url https://robinhoodchain.blockscout.com/api \
  --constructor-args $(cast abi-encode "constructor(address)" $HOOK)
```

Se o comando falhar (verificadores às vezes são temperamentais): use a UI do Blockscout — página do contrato → “Verify & Publish” → “Via Standard JSON Input”, e envie o arquivo gerado por `forge verify-contract ... --show-standard-json-input > hook.json`. Compilador: 0.8.26, otimizador ligado, 800 runs, via-IR ligado (conferir com o `foundry.toml`).

✓ Critério de sucesso

As páginas dos dois contratos no Blockscout mostram o selo “Verified” com o código legível.

Parte 4 Seed: o lançamento (irreversível)

⚠ Ponto de não retorno

Esta única transação: (1) inicializa a pool ETH/QUIVER com o preço da Parte 2; (2) deposita **100% do supply** como liquidez unilateral em QUIVER; (3) **renuncia a ownership** — depois dela não existe mais nenhum controle privilegiado sobre o contrato, por design. Confira `TICK_UPPER` e `TICK_LOWER` no `.env` **duas vezes** antes de executar.

4.1 — Checklist pré-seed

- ☐ Partes 1–3 completas, todos os critérios verdes
- ☐ `H00K` preenchido no `.env` e `source .env` re-executado
- ☐ `TICK_UPPER` / `TICK_LOWER` conferidos contra a tabela da Parte 2
- ☐ Contratos verificados no Blockscout (3.4)
- ☐ Site pronto para publicar logo em seguida (Parte 5) — ideal: pool não ficar “anônima” por horas

4.2 — Executar

```
forge script script/Seed.s.sol:SeedQuiver \
  --rpc-url $RPC_URL --broadcast --private-key $PK
```

O que o script faz: valida os ticks (ordem e múltiplos de 200), calcula o `sqrPriceX96` do tick superior e a liquidez exata para couber os 4663 QUIVER no range, e chama `hook.seed(...)` — que inicializa a pool e mint a posição LP via multicall no PositionManager, e renuncia a ownership ao final.

```
sqrPriceX96: ...
liquidity: ...
Seeded. POSM position tokenId: N
Owner after seed (should be 0): 0x0000000000000000000000000000000000000000
```

4.3 — Confirmar on-chain

```
cast call $H00K "seeded()(bool)" --rpc-url $RPC_URL
true
cast call $H00K "owner()(address)" --rpc-url $RPC_URL
0x0000000000000000000000000000000000000000
```

✓ Critério de sucesso

`seeded == true` e `owner == 0x0`. **O Quiver está no ar.** Registre em `docs/DEPLOYMENTS.md`: endereços, tokenId da posição, hash da tx de seed (este hash é a *prova pública do fair launch* — vai na thread de lançamento).

4.4 — Compra de fumaça (prova de vida)

1. Abra `app.uniswap.org`, conecte na Robinhood Chain;
2. No swap, cole o endereço do `H00K` como token de saída (é o próprio QUIVER);
3. Compre um valor que resulte em **mais de 1,0 QUIVER inteiro** (para mintar um Arrow);
4. No Blockscout, abra a página do **mirror** e confirme o evento `Transfer` (mint do Arrow #1 para o seu endereço);
5. Bonus: chame `tokenURI(1)` na aba “Read” do mirror verificado e cole o data-URI no navegador — a arte deve renderizar.

✓ Critério de sucesso

swap ok, Arrow mintado e visível no explorer.

Parte 5 Publicar o site

5.1 — Build com os endereços reais

```
cd ../web
npm install
VITE_HOOK_ADDRESS=$HOOK \
VITE_MIRROR_ADDRESS=$MIRROR \
npm run build
✓ built in ~3s → saída em web/dist/
```

O `dist/` é 100% estático com caminhos relativos — serve em qualquer host, subpasta ou gateway IPFS. Com os endereços presentes, o site sai do modo “Pre-launch” e liga os contadores on-chain, o painel My Quiver e os links do explorer.

5.2 — Opção A: Cloudflare Pages (recomendada)

1. dash.cloudflare.com → Workers & Pages → Create → Pages → Connect to Git → selecione o repo;
2. Build settings: **Root directory** = `web` · **Build command** = `npm run build` · **Output** = `dist` ;
3. Environment variables: `VITE_HOOK_ADDRESS` e `VITE_MIRROR_ADDRESS` ;
4. Deploy. Domínio custom opcional em “Custom domains”.

Vercel/Netlify: mesmos 4 passos com nomes ligeiramente diferentes.

5.3 — Opção B: GitHub Pages

```
cd web
npm run build
# publica dist/ no branch gh-pages
git checkout --orphan gh-pages && git rm -rf .
cp -r dist/* . && git add -A && git commit -m "site" && git push origin gh-pages
```

Depois: Settings → Pages → branch `gh-pages` . (Ou use uma GitHub Action de deploy de Pages.)

5.4 — Opção C: IPFS + ENS (estilo Prism)

1. Suba o `dist/` para um pinning service:

```
# com Pinata (UI): arraste a pasta dist/ em "Upload → Folder" e copie o CID
# ou com ipfs local:
ipfs add -r web/dist
... added <CID> dist
```

2. No app do ENS (app.ens.domains), no seu nome → Records → **Content Hash** = `ipfs://<CID>` ;
3. Acesse em `https://<seunome>.eth.limo`.

A cada atualização do site: novo upload → novo CID → atualizar o Content Hash.

5.5 — Smoke test do site no ar

- ☐ Hero mostra “Live arrows” > 0 (após a compra de fumaça) e status “Live”
- ☐ “Connect” abre a carteira e adiciona/troca para a chain 4663
- ☐ “My Quiver” lista seu Arrow **com a arte SVG renderizada** e fees pendentes
- ☐ “Claim all fees” dispara transação (pode dar valor ~0 no começo — normal)
- ☐ Links de contrato abrem o Blockscout nas páginas verificadas
- ☐ “Buy QUIVER ↗” abre o Uniswap com o token pré-selecionado

Parte 6 Pós-lançamento (primeiras 48h)

6.1 — Testar o fluxo de fees de ponta a ponta

```
# depois de existir algum volume de swap:  
cast send $H00K "pokeFees()" --rpc-url $RPC_URL --private-key $PK  
# então claim no site (My Quiver → Claim all fees) e conferir o ETH chegando
```

6.2 — Metadata do token no Blockscout

Página do token → “Token info” → submeter logo, descrição e site (formulário do próprio explorer; usa a conta Blockscout).

6.3 — Tornar o repositório público + thread de lançamento

Código legível é parte do marketing (foi assim no Prism). Modelo de thread (adapte):

```
1/ Quiver is live on Robinhood Chain.  
   one v4 pool. 4663 arrows. holding is providing liquidity.  
  
2/ Every whole QUIVER you hold auto-mints one Arrow NFT –  
   a 1/4663 share of the single Uniswap v4 LP position.  
   Fees from every swap stream to arrows pro-rata. No staking. No wrapper.  
  
3/ Fair launch, provably: 100% of supply seeded one-sided,  
   zero team allocation, ownership renounced in the seed tx itself:  
   <link da tx de seed no Blockscout>  
  
4/ Sell a fraction and the arrow burns forever. 4663 is a ceiling,  
   not a floor – every burn makes every remaining arrow's slice bigger.  
  
5/ Site: <url> · Contract: <hook> · Pool: Uniswap v4 · Chain: 4663  
   Art is 100% on-chain. DYOR, no promises, just mechanics.
```

6.4 — Keeper opcional de `pokeFees()`

Suaviza o custo de gás do primeiro comprador após períodos parados (qualquer um pode chamar):

```
# crontab -e (a cada 6h; usar uma carteira qualquer com uns trocados de gás)  
0 */6 * * * cast send 0xH00K "pokeFees()" --rpc-url https://rpc.mainnet.chain.robinhood.com -  
-private-key 0x... >/dev/null 2>&1
```

6.5 — Listagens

- **DEX Screener:** indexa a pool automaticamente após os primeiros trades;
- **CoinGecko / CMC:** formulário de listagem após tração (volume/holders consistentes);
- **Marketplaces NFT** que suportem a chain: a coleção é o endereço do mirror.

Apêndice A Solução de problemas

Sintoma	Causa provável	Solução
<code>bootstrap.sh</code> falha no clone	Rede/GitHub momentâneo	Rodar de novo — retoma de onde parou
<code>preflight.sh</code> : chain id ≠ 4663	RPC de outra rede no ambiente	Conferir <code>RPC_URL</code>
<code>preflight.sh</code> : “no code”	Endereço errado	Reconferir DEPLOYMENTS.md / fonte oficial Uniswap
Fork test FAIL no seed	Interface do POSM difere da esperada	Não deployar. Reportar a saída — precisa de ajuste de código
Deploy: <code>HookAddressNotValid</code>	Deploy manual fora do script / salt errado	Usar sempre <code>Deploy.s.sol</code> (minera o salt certo)
Deploy: <code>hook address mismatch</code>	Outro deploy usou o salt antes (raríssimo)	Rodar de novo — novo salt
Seed reverte: <code>spacing / range</code>	Ticks não múltiplos de 200 ou invertidos	Corrigir <code>TICK_LOWER</code> / <code>TICK_UPPER</code>
Seed reverte: <code>Unauthorized</code>	<code>PK</code> não é do <code>HOOK_OWNER</code>	Usar a carteira do deploy
Verificação Blockscout falha	Metadados de compilação divergem	Via UI com Standard JSON (<code>--show-standard-json-input</code>)
Site: contadores em “...”	RPC público lento/bloqueando CORS	Setar <code>VITE_RPC_URL</code> para um endpoint dedicado (Alchemy/QuickNode)
Site: sem Arrows após compra	Comprou < 1,0 QUIVER inteiro	Esperado — NFT só para unidades inteiras
Compra de fumaça não mint Arrow	idem acima, ou swap noutra pool	Conferir que o par usa o hook (fee 1%)

Apêndice B Cartão de referência rápida

Item	Valor
Chain / ID	Robinhood Chain · 4663
RPC	https://rpc.mainnet.chain.robinhood.com
Explorer	https://robinhoodchain.blockscout.com
PoolManager (v4)	0x8366a39CC670B4001A1121B8F6A443A643e40951
PositionManager (v4)	0x58daec3116aae6D93017bAAea7749052E8a04fA7
Permit2	0x00000000000022D473030F116dDEE9F6B43aC78BA3
Supply / unidade	4663 QUIVER · 1 token inteiro = 1 Arrow NFT
Pool	ETH nativo / QUIVER · fee 1% · tickSpacing 200 · hook afterSwap
Preço por QUIVER	$1,0001^{(-tick)}$ ETH · recomendado TICK_UPPER=69000, TICK_LOWER=-887200

```
# sequência completa (cartão de comandos)
cd contracts && ./bootstrap.sh && forge build && forge test -vv
./preflight.sh
forge test --match-contract QuiverForkTest --fork-url https://rpc.mainnet.chain.robinhood.com
-vv
cp .env.example .env # preencher HOOK_OWNER, PK, ticks
source .env
forge script script/Deploy.s.sol:DeployQuiver --rpc-url $RPC_URL --broadcast --private-key $P
K
# colar HOOK/MIRROR no .env, source .env, verificar no Blockscout (3.4)
forge script script/Seed.s.sol:SeedQuiver --rpc-url $RPC_URL --broadcast --private-key $PK
cd ../web && npm install
VITE_HOOK_ADDRESS=$HOOK VITE_MIRROR_ADDRESS=$MIRROR npm run build
# publicar web/dist/ (Pages / gh-pages / IPFS+ENS)
```

QUIVER · one v4 pool. 4663 arrows. · Guia gerado a partir do repositório Pinguim306/v4-hook — docs de referência: GUIA-OPERADOR.md, LANCAMENTO.md, DEPLOYMENTS.md. No espírito do Prism (0xsolazy). Não é aconselhamento financeiro.